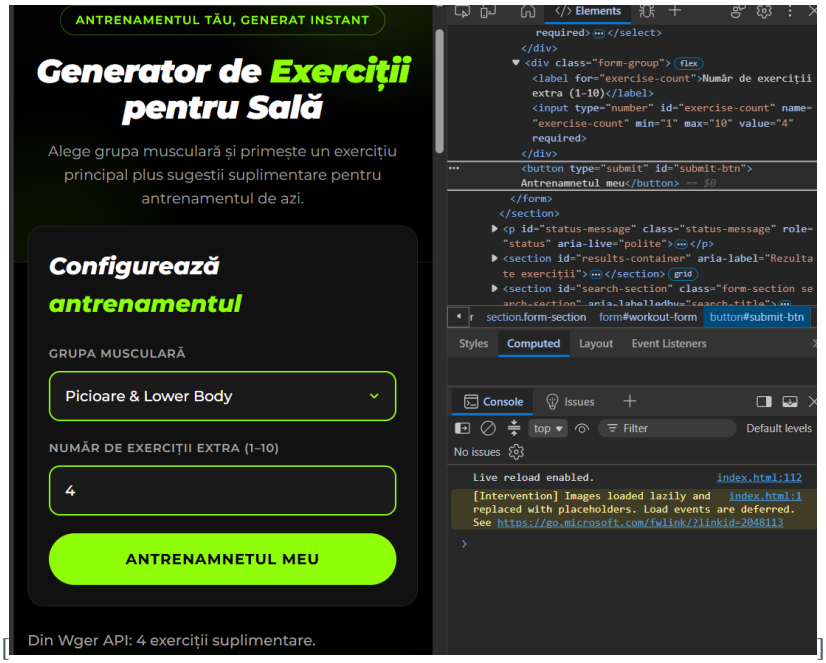
Raport – Analiza aplicației cu Developer Tools

Tehnologii Web Avansate | Laborator de 80 de minute

Date student	De completat
Nume și prenume	Salaghe Alexandru-Ilie
Aplicație / API folosit	Generator de Exerciții pentru Sală – Wger Workout Manager API (https://wger.de/api/v2)
Repository GitHub	https://github.com/alexsalaghe/Aplicatie-Web--Generare-de-exerctitii-pentru-sala
Browser / mod de rulare	

Instrucțiuni: Înlocuiți marcajele [Captură] cu capturile făcute în timpul laboratorului. Lăsați sub fiecare 1–2 propoziții care explică ce se vede și ce ați constatat. Nu includeți informații sensibile.

1. Modificarea DOM în Elements



Captura arată butonul din pagină cu textul modificat și nodul corespunzător selectat în panoul Elements. Modificarea este temporară.

Elementul modificat

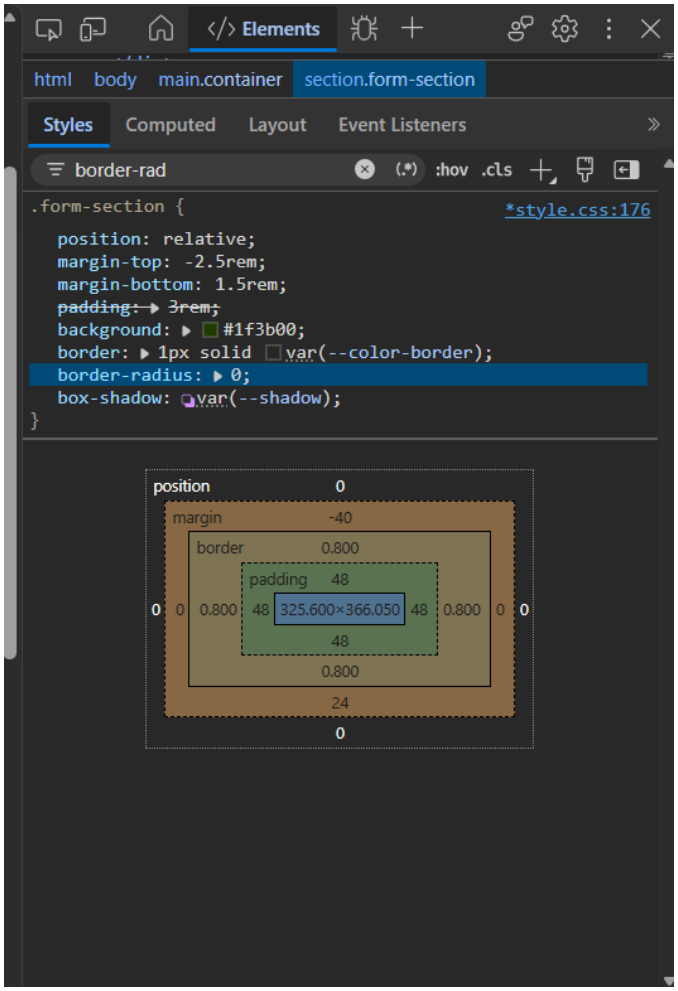
Butonul de submit <button id="submit-btn">; am schimbat textul din „Generează exerciții” în „Antrenamentul meu”.

Ce s-a întâmplat după reload?

Modificarea a dispărut. Elements modifică doar DOM-ul din memoria browserului, nu fișierul index.html; la reload browserul reîncarcă pagina din fișierul original.

2. Styles / Computed / Box model





Am modificat din Styles padding-ul și fundalul cardului formularului; diagrama din Computed arată cum padding-ul, border-ul și margin-ul formează box modelul elementului. Marginea negativă de sus face cardul să urce peste header.

Proprietăți CSS modificate

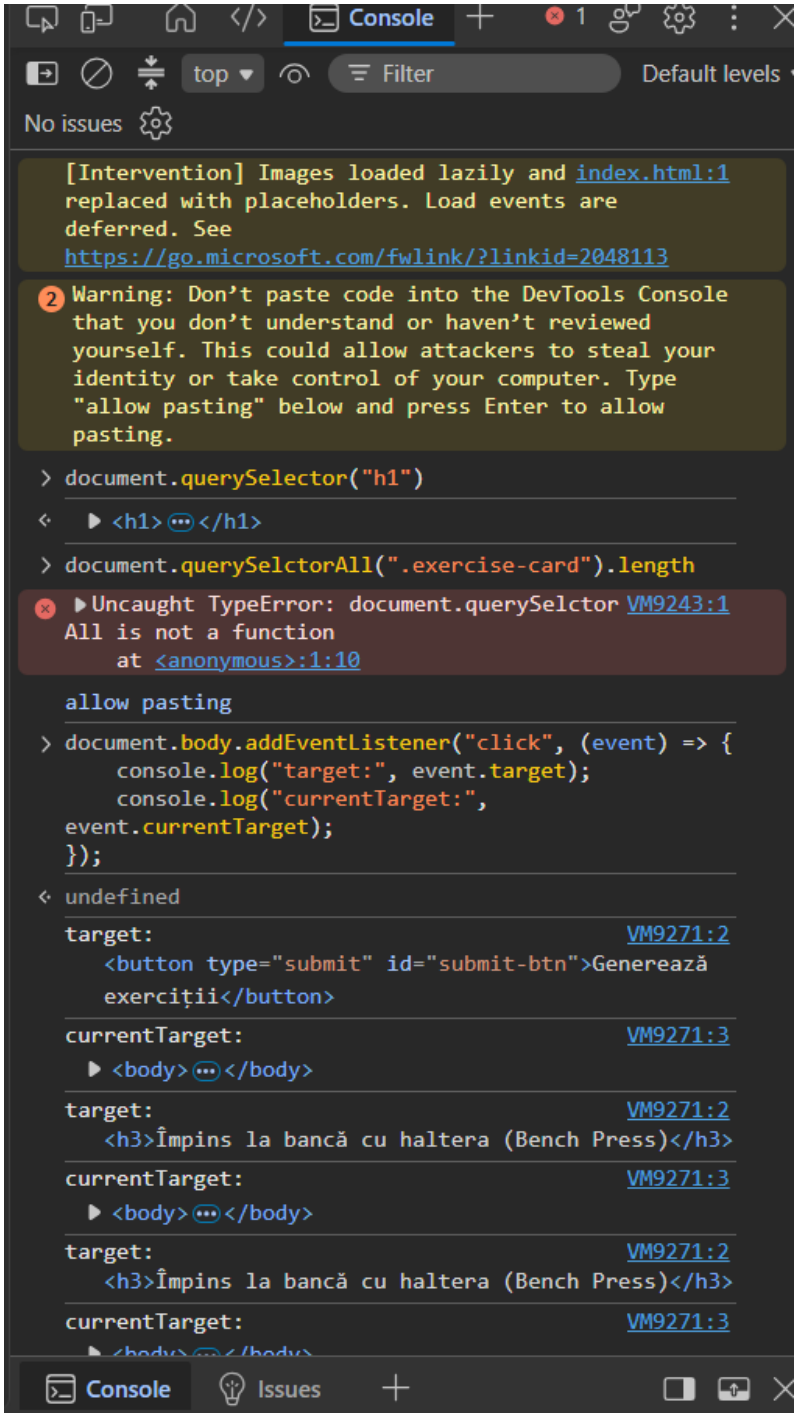
Pe .form-section: padding 1.4rem → 3rem, background var(--color-surface) → #1f3b00 (și border-radius 18px → 0).

Ce valori ați observat pentru padding/border/margin/box-sizing?

padding 48px pe fiecare latură (după modificare; inițial 32px), border 1px, margin -40px sus, 24px jos, 0 lateral, box-sizing: border-box – deci width-ul include padding-ul și border-ul.

3. Console – expresii și evenimente

[CAPTURĂ 3 – lipiți aici imaginea relevantă]



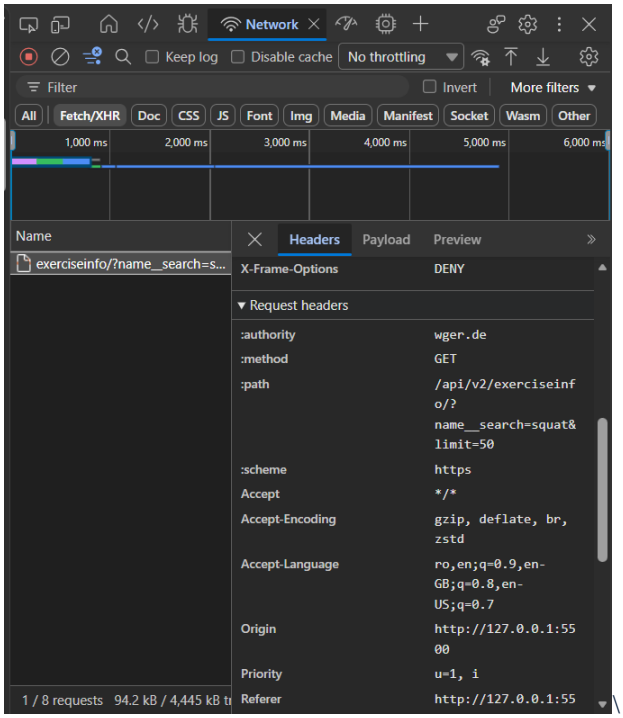
Console arată rezultatul expresiei document.querySelector("h1") și, la click pe buton, target = <button>, currentTarget = <body>.Ce ați testat în Console

Am verificat că nu apar erori la încărcare și la generarea exercițiilor. Am rulat document.querySelector("h1"), care a returnat titlul paginii, apoi am adăugat un listener de click pe body și am apăsat butonul „Generează exerciții” și titlul unui card.

Care este diferența dintre target și currentTarget?

event.target este elementul pe care s-a dat efectiv click (butonul sau <h3>), iar event.currentTarget este elementul care are atașat listenerul (<body>). Ele diferă pentru că evenimentul urcă prin DOM (event bubbling) de la element până la body.

4. Network – cererea către API



Cererea Fetch este de tip GET către Wger API și a primit răspuns 200 OK în format JSON. Header-ul access-control-allow-origin: * permite apelul direct din browser, fără cheie API.

Metodă HTTP :GET

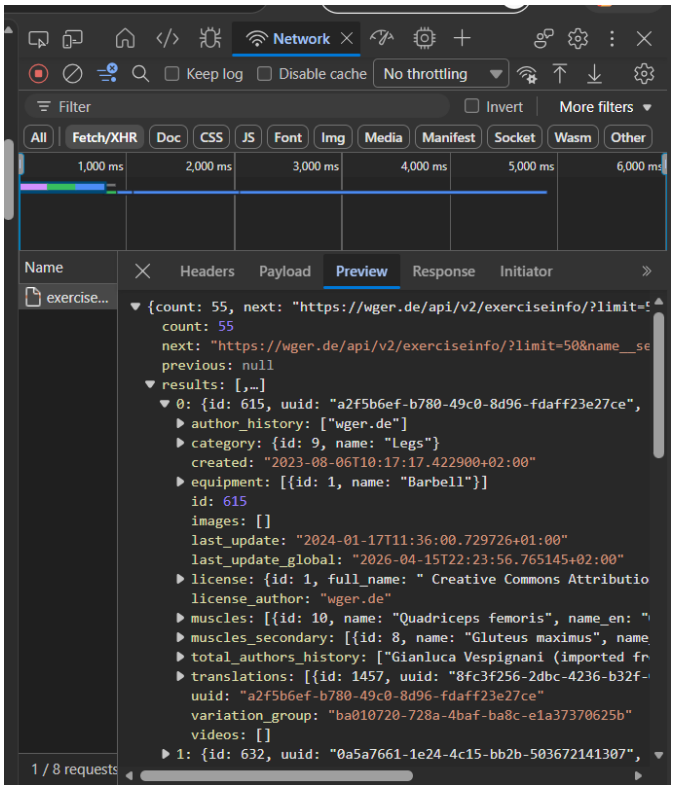
Status HTTP 200 OK

Durata normală (ms) 782ms

Ce a declanșat cererea?

Trimiterea formularului de căutare (evenimentul submit pe #search-form) cu termenul „squat”. Funcția searchExercises() din script.js face fetch() către https://wger.de/api/v2/exerciseinfo/?name__search=squat&limit=50.

5. Network – răspunsul API

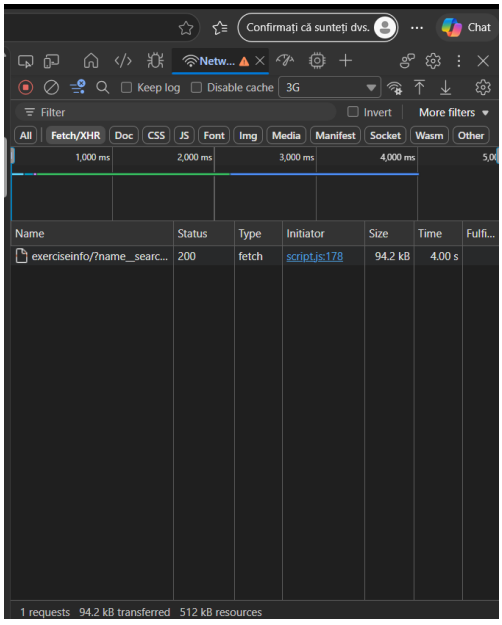


Preview arată răspunsul JSON al API-ului Wger. Aplicația folosește count pentru mesajul de status, iar din fiecare element din results preia numele (translations), imaginea (images), categoria (category) și mușchii (muscles) pentru a construi cardurile.

Câmp JSON 1 și utilizarea sa count (ex. 54) – numărul total de exerciții găsite; este afișat în mesajul de status „Am găsit 55 de exerciții pentru „squat”, afișăm 12.”

Câmp JSON 2 și utilizarea sa results[].translations[].name (ex. „Box squat”) – numele exercițiului; scriptul alege traducerea cu language: 2 (engleză) și îl afișează ca titlu (<h3>) pe cardul rezultatului.

6. Network – conexiune lentă



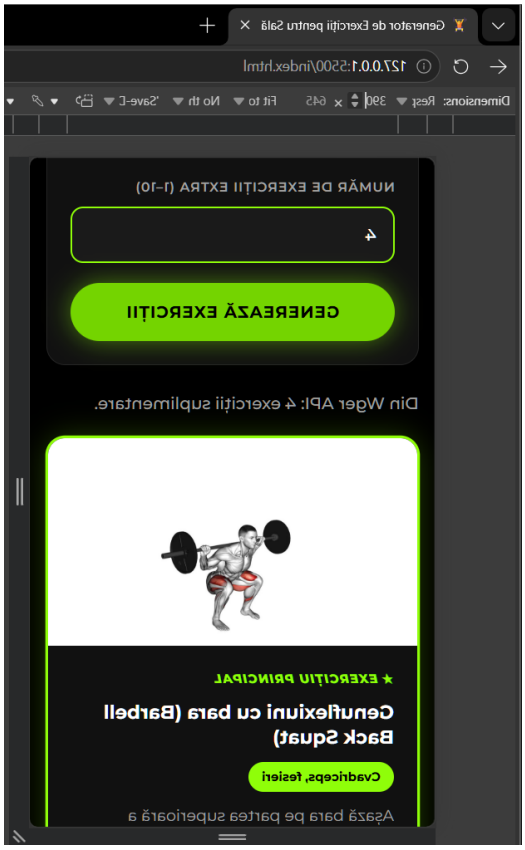
Cu throttling activat (3G), aceeași cerere a durat ~4 s în loc de 782 ms; aplicația afișează un mesaj de încărcare și blochează butonul. În modul Offline aplicația afișează un mesaj de eroare. Dacă API-ul nu ar răspunde deloc, aplicația ar rămâne în starea de încărcare, pentru că fetch() nu are timeout.

Durata cu limitare (ms) ≈ 4000 ms (4 s)

Diferența față de testul normal : Cu limitarea activată, cererea a durat ~4000 ms față de 782 ms fără limitare, adică de aproximativ 5 ori mai mult. Diferența vine din viteza redusă de transfer: răspunsul JSON are ~500 KB.

Afișează aplicația loading? Da. Apare mesajul „Se caută „squat”...” și butonul Caută este dezactivat până la primirea răspunsului. Nu există însă un spinner sau un skeleton vizual.

7. Responsive – Device Mode

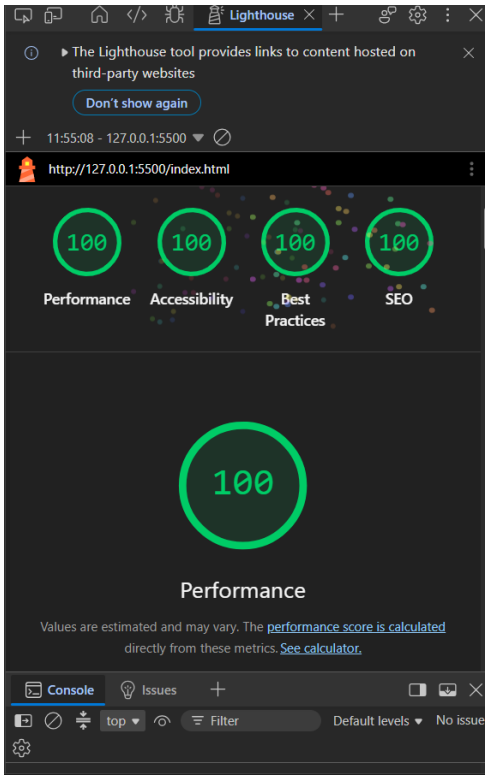


Lățimea de test 390 px (iPhone 12 Pro, 390 × 844).

Problemă observată sau motivul pentru care layoutul este utilizabil

Layoutul se adaptează corect: sub 600 px media query-ul pune formularul pe o coloană, butonul pe toată lățimea și cardurile pe o singură coloană. Nu apare scroll orizontal.

8. Lighthouse – audit



Performance 100

Accessibility 100

Best Practices 100

SEO 100

Recomandarea aleasă

„Image elements do not have explicit width and height” (CLS – Cumulative Layout Shift). Auditul pe încărcarea inițială a obținut 100, dar nu a analizat cardurile generate după trimiterea formularului; imaginile acestora, preluate din Wger API, sunt create din JavaScript fără atributele width și height.

Cum ați remedia-o?

În funcția createCardImage() din script.js aș seta img.width și img.height (ex. 340 × 200) pe fiecare imagine creată, astfel încât browserul să rezerve spațiul înainte ca imaginea să se încarce și cardurile să nu „sară” în pagină. Suplimentar, aș redimensiona pozele locale la ~800 px lățime și le-aș converti în WebP, pentru a reduce dimensiunea descărcată.

Concluzie (max. 5 rânduri)

Analiza cu Developer Tools a arătat că aplicația funcționează corect: nu apar erori în Console, cererile către Wger API sunt de tip GET și răspund cu 200 în ~780 ms, iar layoutul se adaptează fără scroll orizontal la 390 px și 768 px. Lighthouse a acordat 100 la toate categoriile pentru încărcarea inițială. Pe conexiune lentă (3G) aceeași cerere a durat ~4 s, iar aplicația afișează doar un mesaj text de încărcare. Primul lucru pe care l-aș îmbunătăți în cod este tratarea cererilor lente: un indicator vizual de încărcare (spinner) și un timeout cu AbortController, astfel încât utilizatorul să primească un mesaj de eroare dacă API-ul nu răspunde.

Predare

Salvați acest document în docs/ din repository-ul proiectului, efectuați commit și push, apoi transmiteți linkul repository-ului.